

Testarea Sistemelor Software

Beldi Darius Vlad - 342

Ivan Emil Ovidiu - 342

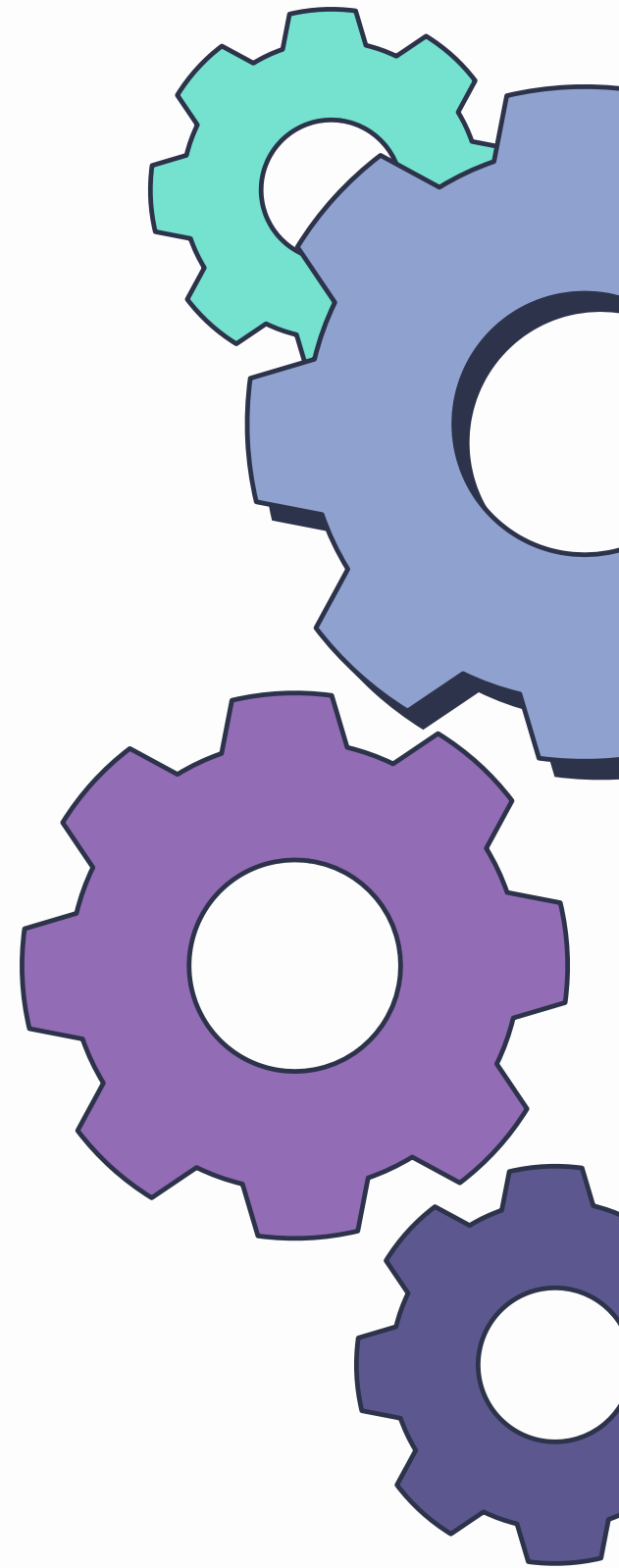


Project Description

- BankingApp is a simple Java console application for simulating basic banking functionalities, including user registration and login, card management, money transfers, and transaction history. It uses a MySQL database for persistent storage and logs actions for auditing purposes.

Features

- User Registration & Login: Create an account or log in with your credentials.
- Card Management: Create and view cards associated with your account.
- Money Transfers: Transfer money between cards, including to contacts in your address book.
- Transaction History: View incoming and outgoing transactions for your cards.
- Address Book: Manage a list of contacts for easy transfers.
- Audit Logging: Every action is recorded in an audit log for traceability.
- Database Initialization: Option to initialize and populate the database with default data.



Functional Testing

Am testat 2 functii :

1) Se testează o funcție care primește un șir de caractere și returnează hash-ul său MD5 sub forma unui șir hexadecimal de lungime fixă. Funcția acceptă orice String nenul ca intrare validă și produce un rezultat determinist. Dacă șirul de intrare este null, funcția nu îl poate procesa.

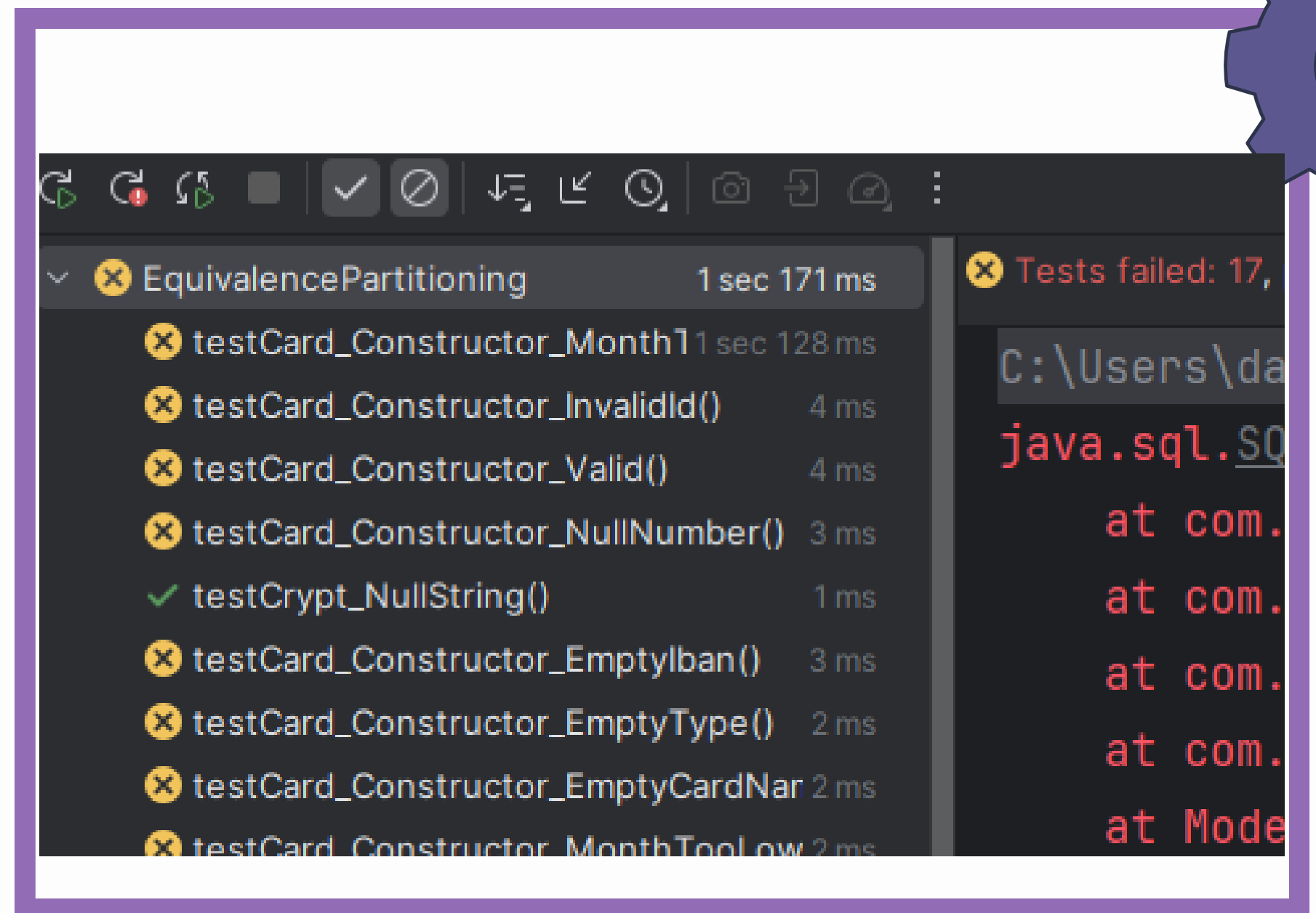
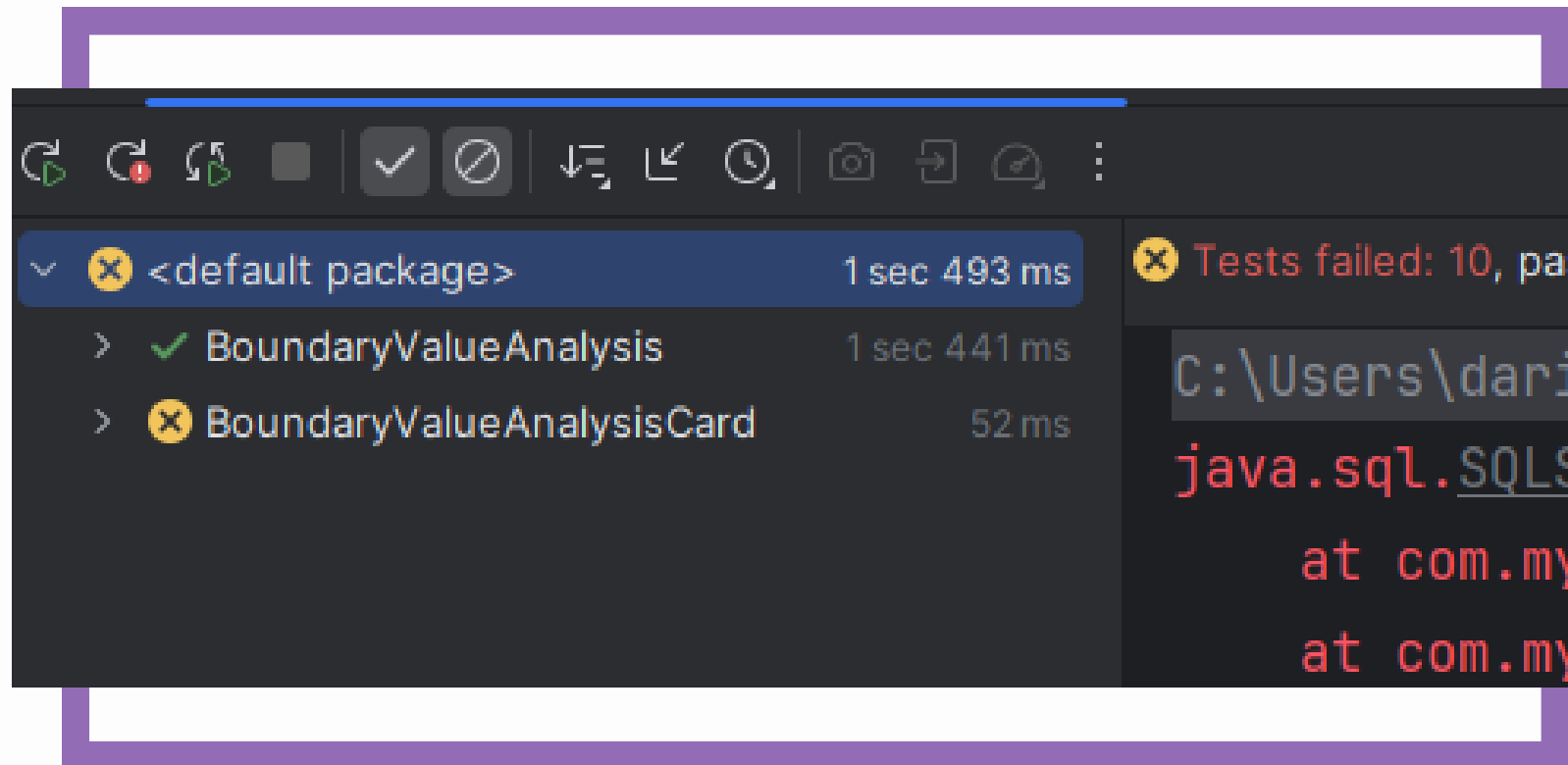
→ **public static String Crypt(String input) throws NoSuchAlgorithmException**

2) Se testează constructorul clasei Card, care creează un obiect reprezentând un card bancar asociat unui utilizator. Constructorul primește 10 parametri ce descriu identitatea, datele și limitele cardului. Un card valid trebuie să respecte constrângerile fiecărui parametru în parte; dacă oricare dintre parametri se află în afara domeniului valid, obiectul nu poate fi creat și constructorul aruncă `IllegalArgumentException`.

→ **Card(int id, int userId, String type, String cardName, String iban, String number, int month, int year, int cvv, int limit)**



Functional Testing 2

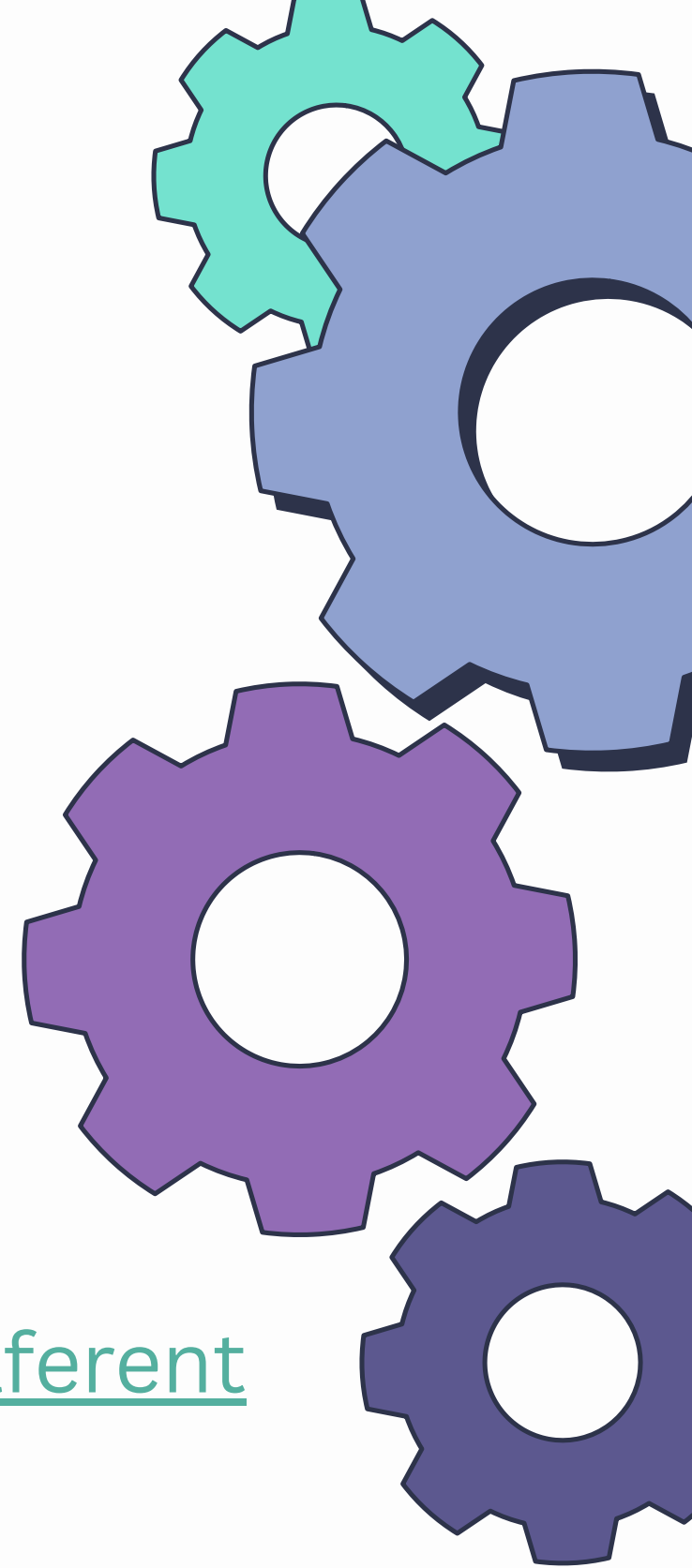


Structural Testing

Functia de testat:

```
1  public User login(String email, String password) throws SQLException, NoSuchAlgorithmException {
2      checkForExistingEmailStatement.setString(1, email);
3      ResultSet rs = checkForExistingEmailStatement.executeQuery();
4      String foundEmail = "";
5      if(rs.next()){
6          foundEmail = rs.getString(1);
7      }
8
9      if(foundEmail.equals(email)){
10         getPasswordStatement.setString(1, email);
11         ResultSet passwordResult = getPasswordStatement.executeQuery();
12         if(passwordResult.next()){
13             if(passwordResult.getString(1).equals(Crypt(password))){
14                 getUserStatement.setString(1, foundEmail);
15                 ResultSet rsUser = getUserStatement.executeQuery();
16                 if(rsUser.next()){
17                     int id = rsUser.getInt(1);
18                     String firstName = rsUser.getString(2);
19                     String lastName = rsUser.getString(3);
20                     Date birthDate = rsUser.getDate(4);
21                     String userEmail = rsUser.getString(5);
22                     String userPassword = rsUser.getString(6);
23
24                     auditService.logAction("Logged in successfully for user: " + id);
25                     return new User(id, firstName, lastName, birthDate, userEmail, userPassword, true, new ArrayList<>());
26                 }
27             }
28         }
29     }
30     return null;
31 }
```

CFG-ul aferent



Structural Testing 2

1) Instruction Coverage

testLoginSuccess() tests almost all the lines (doesn't make sense to test for the else, because they all lead to return null)

2) Decision Coverage

The tests :

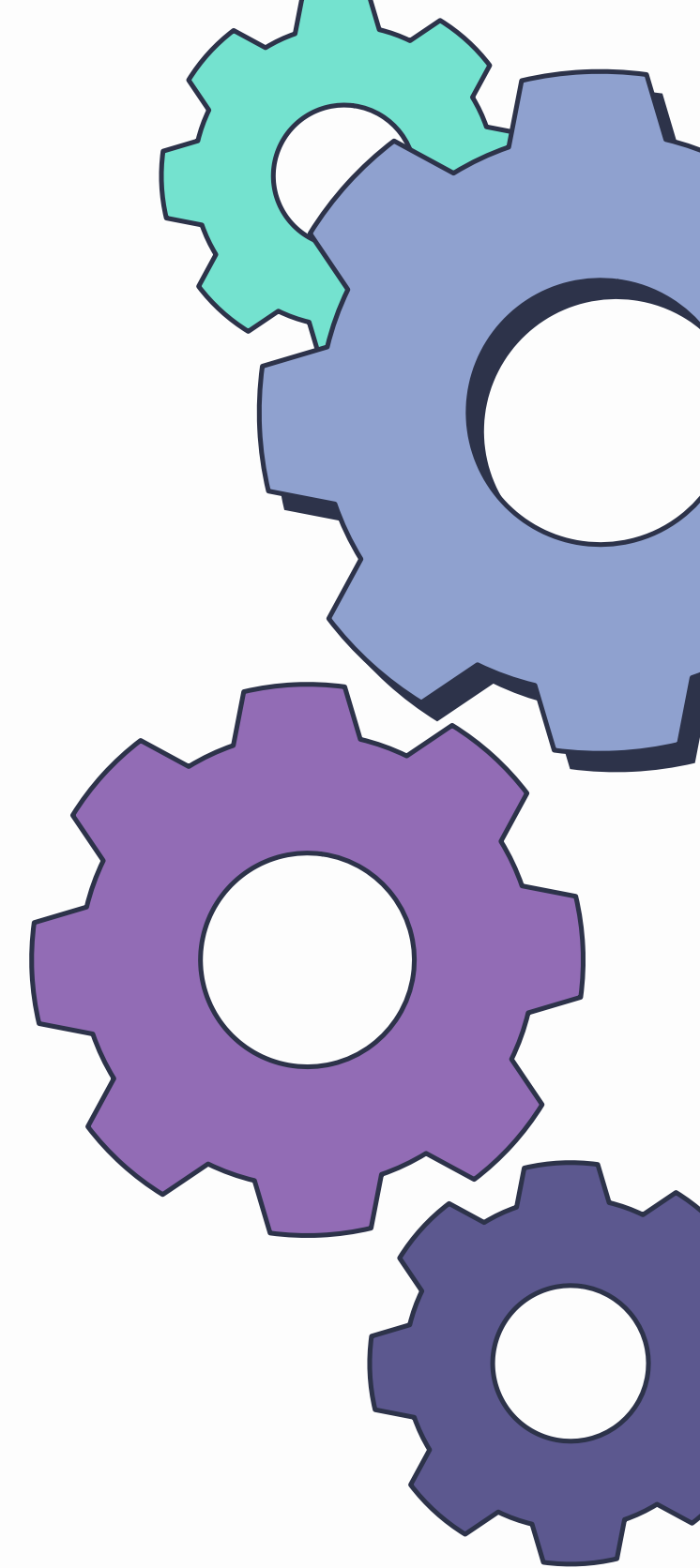
- testLoginEmailNotFound()
- testLoginWrongPassword()
- testLoginUserNotFoundAfterPasswordMatch()
- testLoginPasswordResultNotAvailable()

All those test for when the if clauses are false and go to line 30, return null

3) The conditional coverage tests are the following :

- testIsNullAllFieldsPresent()
- testIsNullFirstNameNull()
- testIsNullLastNameNull()
- testIsNullBirthDateNull()
- testIsNullEmailNull()
- testIsNullPasswordNull()

Structural Testing 3



Complexitatea ciclomatica

$$V(G) = E - N + 2 = 14 - 10 + 2 = 6$$

Setul de baza contine 6 circuite independente.

Setul de baza

#	Circuit	Cazul testat
a	N1, N2, N3, N4, N5, N6, N7, N8, exit	login reusit
b	N1, N2, N4, N9, exit	email inexistent
c	N1, N2, N3, N4, N9, exit	email gasit dar nu corespunde
d	N1, N2, N3, N4, N5, N9, exit	nu exista parola in tabela
e	N1, N2, N3, N4, N5, N6, N9, exit	parola gresita
f	N1, N2, N3, N4, N5, N6, N7, N9, exit	parola corecta dar user inexistent

Mutations

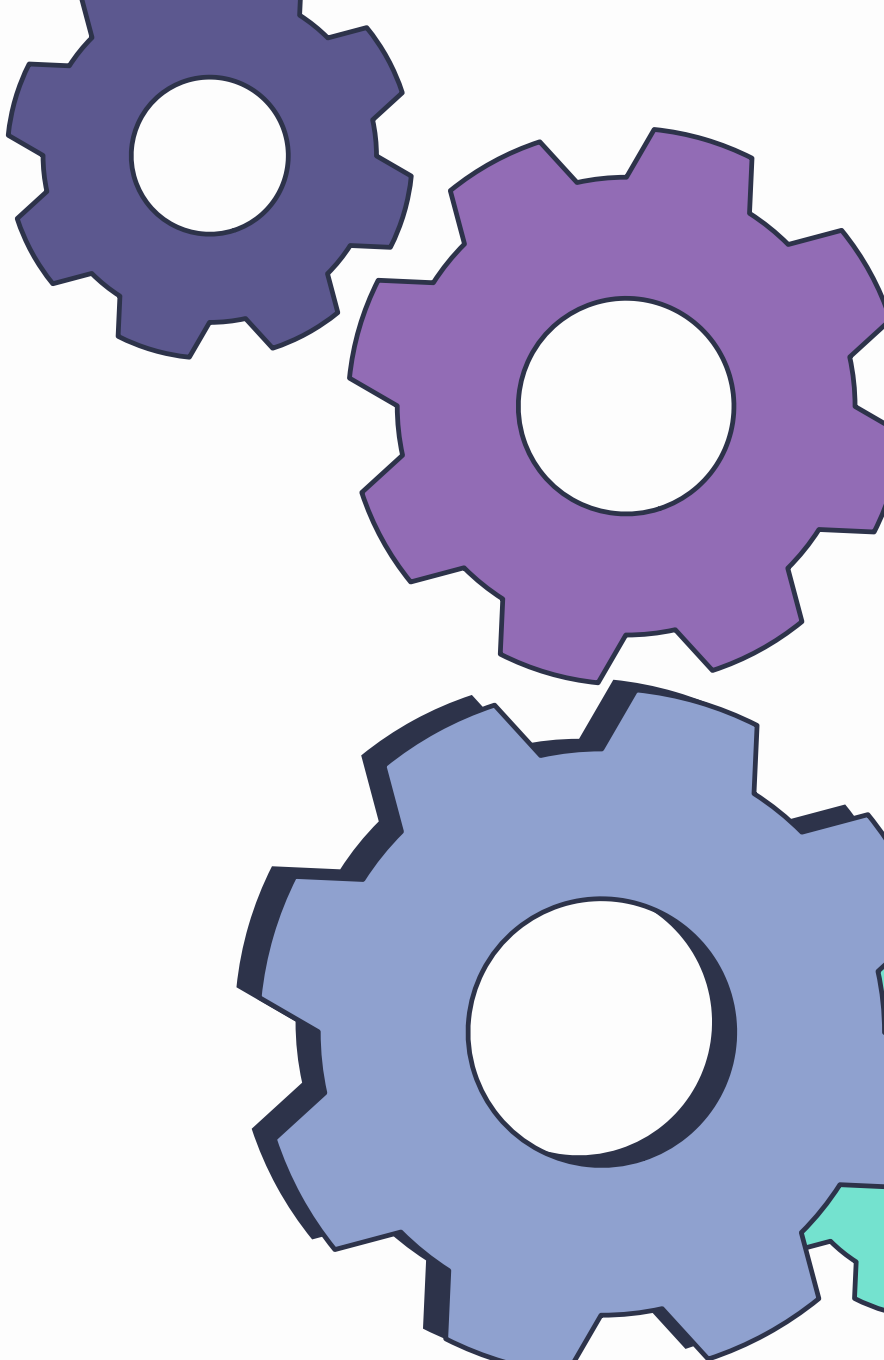
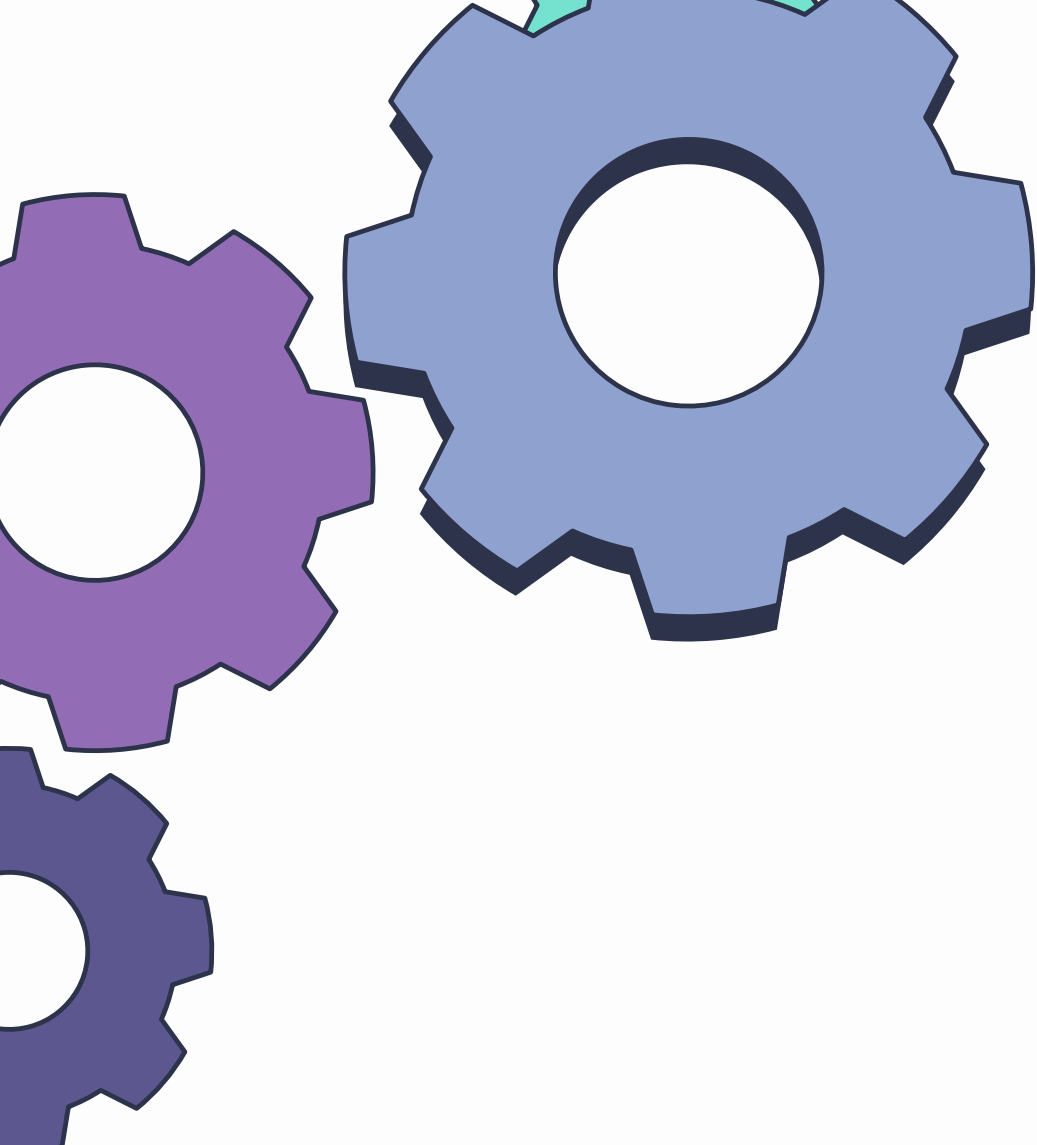
Am testat pe functia:

```
public static boolean isNull(String firstName, String lastName, Date birthDate,  
String email, String password) {  
    if (firstName == null || lastName == null || birthDate == null  
        || email == null || password == null)  
        return true;  
    else  
        return false;  
}
```

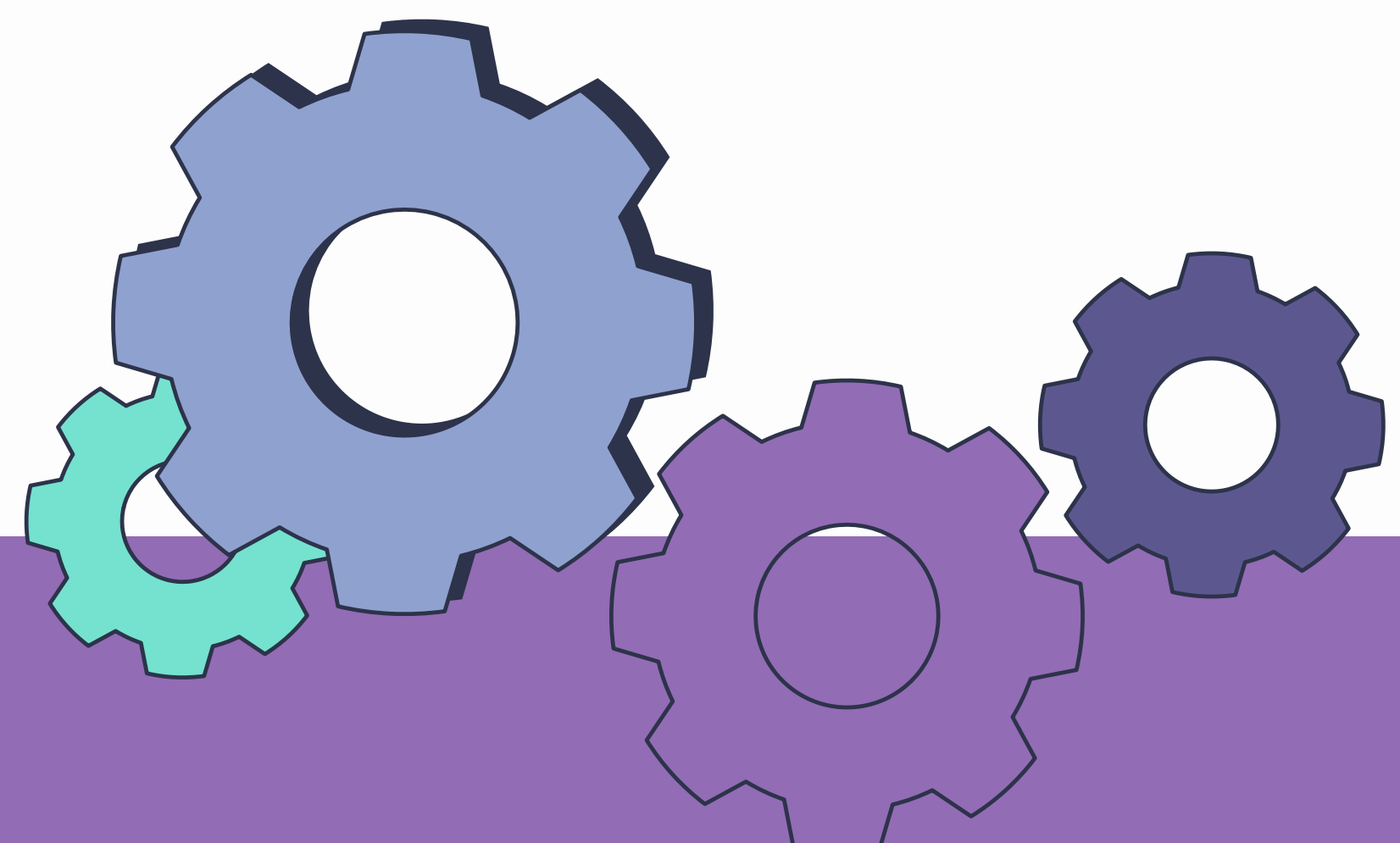


Raport PIT final

Metrica	Inainte	Dupa
Mutanti generati	12	12
Omorati	9	11
Supravietuitori	3	1
Mutation Score	75%	92%

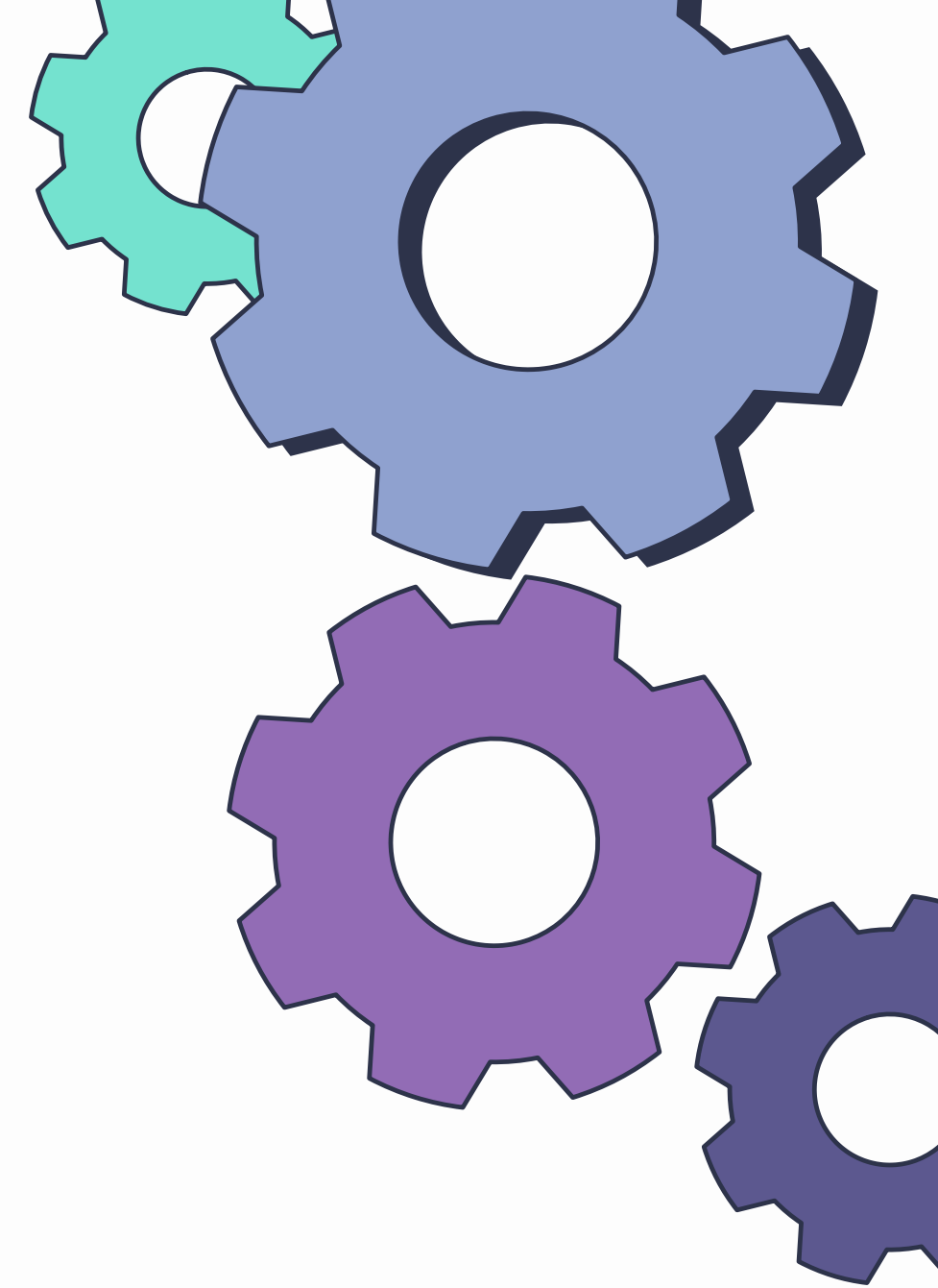


Raport AI



RESULTS!!!!

java in BankingApp: 75 total, 27 failed, 48 passed



Multumim

